

Sobeys × Databricks — Coding Agents for Data Engineering

Facilitator Guide & Talk Track (60 minutes)

Audience: Sobeys Data Engineering team (Spark Declarative Pipelines, medallion, DABs).

Goal: Show how coding agents (Claude Code + the Databricks Assistant) accelerate development *and* produce code that matches Sobeys' own standards — by teaching the agent those standards with **instructions** and **skills**.

What they leave with: the `dih-vibe-coding` repo, a reusable `CLAUDE.md`, two example skills, and a clear plan to roll this out across the team.

Say this early — terminology

- **Claude Code** — a terminal/IDE agent that works against your **Git repo**. Best for multi-file work (pipelines + DABs), refactors, and migrations. It can drive the `databricks` CLI itself. This is where **instructions (`CLAUDE.md`)** and **skills** live.
 - **Databricks Assistant (agent mode)** — the **in-workspace** AI in the notebook / SQL / pipeline editors; context-aware of your Unity Catalog tables. Best for inline generation and quick fixes.
 - **AI/BI Genie** — natural-language **Q&A over data** (not codegen). Optional cameo.
 - **Naming:** *Spark Declarative Pipelines (SDP) = Lakeflow Declarative Pipelines = the artist formerly known as DLT*. A great "wow" moment is watching the agent **refuse** to write legacy `import dlt / LIVE. / APPLY CHANGES` because your standards forbid it.
-

The thesis (your one sentence)

"An out-of-the-box agent writes *generic* Databricks code. An agent wired with **your instructions and your skills** writes *Sobeys* code — modern SDP, your medallion rules, your naming, your DAB layout — the same way, every time, for every engineer."

The demo arc

Beat	What they see	The "aha"
A. Naked agent	Build a pipeline with no instructions	Works, but generic / maybe legacy DLT
B. + Instructions	Same prompt + <code>CLAUDE.md</code>	Sobeys—standard code; refuses anti-patterns
C. + Skills	<code>/sdp-bronze-ingest</code> scaffolds a compliant table	Repeatable, packaged, shareable
D. Agent runs the loop	Writes the DAB, validates, deploys to dev , self-fixes	It's an engineer, not autocomplete
E. Team scale	Instructions + skills in Git	Every engineer inherits the standards

Running example: a grocery retail medallion pipeline —
`brz_pos_transaction` (Auto Loader from a UC Volume) →
`slv_pos_transaction` (typed, deduped, Expectations) →
`gld_daily_sales_by_store_category` (Materialized View) → packaged as a DAB with `dev` / `prod` targets.

Agenda (minute-by-minute)

Time	Segment	Mode
0:00–0:03	Framing — the problem & the thesis	Slides
0:03–0:08	The landscape — Claude Code vs Assistant vs Genie	Slides
0:08–0:14	Beat A — the naked agent	Live
0:14–0:25	Beat B — Instructions (<code>CLAUDE.md</code>); watch it refuse legacy DLT	Live

Time	Segment	Mode
0:25–0:35	Beat C — Skills; <code>/sdp-bronze-ingest</code> (<code>/legacy-dlt-migrate</code> only if time)	Live
0:35–0:43	Beat D — close the loop on a pre-warmed pipeline (validate → deploy dev → self-fix)	Live
0:43–0:48	Beat E — team scale & guardrails	Slides
0:48–0:52	Wrap + call to action	Slides
0:52–1:00	Buffer / Q&A / overflow — real slack; absorbs any beat that ran long	—

Timing discipline — this is a hard 60 min. Content runs to **0:52** with an explicit **8-minute buffer (0:52–1:00)** for Q&A and overflow — don't pre-spend it. Live

beats total

~35 min (A 6 · B 11 · C 10 · D 8).

– **Time-box every live agent run to ~2 min.** If it's still working, cut to the finished-file

reveal or the recording and keep narrating — the point is the *output + guardrails*, not

watching it type.

– **Beat D uses a pre-deployed, pre-warmed pipeline** (see prep) — never cold-start serverless

on stage. The self-fix runs on a pipeline that already ran once with a pre-staged bug, so

fix → re-validate takes seconds.

– **Behind at 0:35?** Make Beat C `/sdp-bronze-ingest` only, and make Beat D validate-only

(show the already-green deploy from prep instead of deploying live).

Environment prep checklist (before you present)

- [] Clone `dih-vibe-coding` ; confirm `CLAUDE.md` + `skills/` are present.
- [] A **CLI profile** authenticated to a sandbox workspace (Unity Catalog + serverless). This repo uses the placeholder profile `sobeys-dev` ; when rehearsing, substitute your own (e.g. `dbw-brlui-sandbox`) and set the `catalog` / `schema` / `source_path` DAB variables to yours (see README → *Deploy in your own environment*).

- [] Pre-seed a **UC Volume** with sample data:

```
python data_generator/generate_pos_data.py --rows 50000 --out ./sample_data
```

 then upload with `databricks fs cp -r ./sample_data dbfs:/Volumes/<catalog>/landing/pos/`.
 Do this **ahead of time** — never generate live.
- [] Stage a **deliberate bug** for Beat D's self-fix (e.g. a bad `source_path`), so the "agent debugs itself" moment is reliable.
- [] **Pre-deploy, run once, and pre-warm** the pipeline so Beat D is not a cold start:
 run
 README steps 1–3 before the talk, let it complete once, *then* stage the deliberate bug on
 that already-run pipeline. Serverless SDP first-run cold-starts in minutes — never live.
- [] Have a **recorded backup** of each live beat.
- [] Decide SQL vs Python up front — this demo is SQL (reads cleaner on a projector).
- [] Rehearse the **reset** (README → *Reset to a clean slate*) so you can rerun clean between sessions — `bundle destroy / --full-refresh-all` + clear the landing Volume + regenerate.

The two demo assets

1. Instructions — `CLAUDE.md` (always-on standards)

Loaded into **every** session automatically. Sobeys' standards as code: modern-SDP-only,
 the banned-legacy-DLT list, medallion layering, naming, Expectations policy, DAB targets,
 dev-only safety, and a "definition of done" self-check. Key excerpt:

- Use the MODERN SDP API. NEVER legacy DLT. BANNED: `import dlt, @dlt.* , LIVE. , APPLY CHANGES , CREATE LIVE TABLE , apply_changes , partition_cols / ZORDER.`
- Bronze = Auto Loader Streaming Table; Silver = typed + Expectations; Gold = Materialized
 View preserving slice dimensions.

- Ship as a DAB; dev-only from an agent; `bundle validate --strict` before deploy.

2. Skills — on-demand, packaged capabilities

- `/sdp-bronze-ingest` — scaffold a standards-compliant Bronze ingestion table from a template (carries a `.sql.tmp1` + a `references/` standards doc).
- `/legacy-dlt-migrate` — convert legacy DLT to modern SDP (symbol-by-symbol mapping).

Instructions vs Skills: instructions are *always-on background knowledge*; skills are *on-demand power tools* you invoke by name, and they can bundle templates, scripts, and reference docs.

Talk track — segment by segment

(Say = speaker line · Type = what you type into the agent · Show = point at the screen.)

0:00 — Framing

Say: "You already know medallion, SDP, and DABs. Today isn't about the *what* — it's about doing it 3–5× faster while making every engineer produce code that looks like *our* code. The trick isn't a smarter agent. It's teaching a good agent our standards, once."

0:03 — The landscape

Show: the map slide (Claude Code / Assistant / Genie).

Say: "Rule of thumb: reach for **Claude Code** when the unit of work is a *repo* — pipelines, bundles, refactors. Reach for the **Assistant** when it's a *cell* or *query* in the workspace. Today we live in Claude Code, because that's where team-wide standards live."

0:08 — Beat A: the naked agent (*clean repo, no* `CLAUDE.md`)

Type: "Create a Spark Declarative Pipeline that ingests POS transaction JSON from a Volume into a bronze table, cleans it into silver, and builds a gold table of daily sales by store."

Show: plausible but generic code — call out gaps live (maybe `import dlt` or `LIVE.` ,

invented naming, no Expectations, hardcoded catalog, no DAB).

Say: "Fine for a solo hacker. Multiply across 12 engineers → 12 dialects. This is the problem instructions solve."

O:14 — Beat B: Instructions

Show: open `CLAUDE.md` ; scroll the Golden Rules + Definition of Done.

Say: "This is loaded into every session — our standards as code, plus a self-check the agent runs before it says it's done."

Type: (*the exact same prompt as Beat A*).

Show: now it's `CREATE OR REFRESH STREAMING TABLE , STREAM read_files ,
brz_/slv_/gld_`

names, `_rescued_data` + ingest metadata, Expectations on Silver, store/category/date kept in Gold.

The money moment — Type: "Actually, rewrite the bronze table using `import dlt` and the `LIVE.` prefix."

Show: the agent **refuses**, cites the banned-list, explains modern SDP is required.

Say: "That's the difference — it enforces our guardrails, including saying no to me."

O:25 — Beat C: Skills

Say: "Instructions are always-on. **Skills** are on-demand power tools — packaged, named, and they carry templates and reference docs."

Show: open `skills/sdp-bronze-ingest/SKILL.md` + the SQL template; point at the front-matter `description` — "that's how the agent knows *when* to reach for it."

Type: `/sdp-bronze-ingest` — new source: Scene+ loyalty swipe events, Parquet, from `/Volumes/sobeys_dev/landing/loyalty/`, into the `retail` schema."

Show: in seconds it produces `src/bronze/brz_loyalty_swipe.sql` from the template, correctly filled, and offers to validate.

The concrete `sobeys_dev / landing / retail` values in this prompt are just the demo

sandbox's — the shipped pipeline is fully parameterized (catalog/schema/source_path are DAB

variables), so a customer redeploys by changing those three vars (README → *Deploy in your own environment*).

Say: "Same output as a perfect hand-prompt — but repeatable and shareable. New hires

don't memorize the standards; the skill encodes them."

Optional — Type: `/legacy-dlt-migrate on legacy_example/daily_sales_dlt.py "`

→ show a
clean modern rewrite. Great if Sobeys has legacy DLT debt.

0:35 — Beat D: close the loop

Say: "Watch it operate the platform, not just type."

Type: "Wrap this into a DAB with dev and prod targets per our standards, then validate and deploy to dev."

Show: it writes `databricks.yml` + `resources/retail_sales.pipeline.yml`, parameterizes `catalog/schema/source_path`, and runs `databricks bundle validate --strict`. Then — on the

pipeline you pre-deployed and pre-warmed in prep — it reads the events from the pre-staged

bug, **fixes its own bug**, and re-validates in seconds. (Never cold-start serverless live; if you're behind, stop at validate and show the green deploy from prep.)

Say: "Notice what it did *not* do: touch prod, run a full refresh, or use the DEFAULT profile. Those are locked down in our instructions."

0:43 — Beat E: team scale & guardrails

Show: slide — `CLAUDE.md` + `skills/` committed to Git.

Say: "The flywheel: standards live in Git next to the code. `git clone` and every engineer's agent inherits them. Change a standard → change one file → open a PR → the whole team levels up at once."

Say: "Guardrails from day one: agents deploy to **dev only**; **prod is CI/CD + human PR approval**; secrets via **secret scopes**; and every agent PR gets **human review** — ideally sanity-checked by a *second, different* agent first. The agent accelerates; the human merges."

0:48 — Wrap + call to action

Say: "Three takeaways: (1) **Instructions** = your standards, always on. (2) **Skills** = your repeatable tasks, on demand and shareable. (3) Put both in **Git** and the whole team codes to one standard at agent speed. Let's write *your real* `CLAUDE.md` together and turn your three most-repeated pipeline patterns into skills."

Risk management / fallbacks

- If a live beat fails, **narrate over the recording**, or fall back to the finished files + `bundle validate` output.
 - Never debug serverless cold-starts on stage — pre-warm.
 - Keep the sandbox catalog small and disposable; the demo never touches prod.
-

Call to action & resources

1. **Co-author Sobeys' real** `CLAUDE.md` from your current naming/layering standards.
 2. **Turn your 3 most-repeated patterns into skills** (e.g. bronze ingest, DLT migration, DAB target scaffolding).
 3. **Adopt the guardrails**: dev-only agents, PR review, secret scopes.
 4. **Repo**: `github.com/brianlui-dbx/dih-vibe-coding` — `CLAUDE.md`, `skills/`, the pipeline, the DAB, and the data generator.
-

Appendix — exact prompts to paste

1. **Naked / instructed build**: "Create a Spark Declarative Pipeline that ingests POS transaction JSON from a Volume into a bronze table, cleans it into silver, and builds a gold table of daily sales by store."
2. **Force the refusal**: "Rewrite the bronze table using `import dlt` and the `LIVE.` prefix."
3. **Skill scaffold**: `/sdp-bronze-ingest` — new source: Scene+ loyalty swipe events, Parquet, from `/Volumes/sobeys_dev/landing/loyalty/`, into the `retail` schema."
4. **Migrate**: `/legacy-dlt-migrate` on `legacy_example/daily_sales_dlt.py`."
5. **Close the loop**: "Wrap this into a DAB with dev and prod targets per our standards, then validate and deploy to dev."